

TRACEFUND

A Transparent and Controlled Digital Government Fund Management System

Competition Project Report

Abstract

TraceFund is a full-stack digital platform designed to improve the transparency, accountability, and controlled utilization of government financial assistance. Traditional welfare and subsidy systems generally transfer funds without enforcing how the money is ultimately spent. This creates opportunities for misuse, unauthorized spending, and difficulties in tracking the flow of public funds.

TraceFund addresses this problem by introducing category-restricted digital funds. Government authorities can allocate funds to beneficiaries under specific categories such as Food, Medicine, and Education. These funds can only be spent at approved vendors belonging to the corresponding category. Every successful transaction is recorded, while audit logs and transparency dashboards provide additional visibility into fund movement.

The system is implemented as a web application using Next.js, React, TypeScript, Python, FastAPI, SQLAlchemy, and SQLite, with a blockchain integration layer designed to support on-chain fund management and verification.

Problem Statement

Government welfare programs distribute large amounts of money for specific purposes such as food assistance, medical expenses, education, and disaster relief. A major challenge is ensuring that the allocated money is actually used for its intended purpose.

Conventional systems may provide limited visibility after funds are distributed. Manual verification can also be time-consuming and may only identify misuse after it has occurred.

The key problems addressed by TraceFund are:

- Lack of control over how allocated funds are spent.
- Difficulty tracking the complete flow of government assistance.
- Possibility of unauthorized or inappropriate transactions.
- Limited transparency for auditors and the public.
- Dependence on manual monitoring and verification.

Proposed Solution

TraceFund converts unrestricted financial assistance into purpose-specific digital funds.

Example:

Government → Beneficiary → ₹10,000 Food Fund → Approved Food Vendor

The beneficiary cannot use the Food Fund at a Medicine or Education vendor. Before a transaction is approved, the system verifies that the beneficiary exists and has the correct role, the vendor exists and is approved, the vendor belongs to the requested fund category, the beneficiary has sufficient balance, and the transaction amount is valid.

If all conditions are satisfied, the transaction is processed and recorded. This creates a rule-based financial ecosystem in which the intended purpose of a fund is enforced by the software itself rather than relying entirely on manual supervision.

Objectives

- Provide controlled digital distribution of government funds.
- Restrict expenditure according to predefined purposes.
- Maintain a traceable transaction history.
- Introduce vendor approval and verification.
- Provide audit capabilities for authorities.
- Provide aggregated financial information to the public.
- Create an architecture capable of integrating blockchain technology.
- Reduce opportunities for misuse through automated validation.

Major User Roles

Government Administrator

- Create and manage fund categories.
- Issue funds to beneficiaries.
- Approve or reject vendors.
- Create government and auditor accounts.
- Monitor system statistics.
- Review audit information.

Beneficiary

- View available fund balances.
- Select an eligible fund category.
- Select an approved vendor.
- Make category-compliant purchases.

- View transaction history.

Vendor

- Register a business and specify its category.
- Participate in transactions after government approval.

Auditor

- Monitor transactions.
- Investigate suspicious activity.
- Review the audit trail.

Public

- View aggregated information such as total funds, spending, transactions, and category-wise utilization without exposing personal information.

System Architecture

TraceFund follows a three-layer architecture:

Frontend → REST API → Database / Blockchain Layer

Frontend:

Next.js, React, and TypeScript provide the user interfaces for different roles and communicate with the backend through HTTP APIs.

Backend:

Python and FastAPI expose REST endpoints for registration, authentication, fund allocation, vendor management, transactions, statistics, audit logs, and blockchain operations. Pydantic is used for data validation and SQLAlchemy provides ORM-based database access.

Database:

SQLite is used as the current database, with SQLAlchemy managing persistence for users, wallets, fund types, vendors, transactions, and audit logs.

Blockchain Layer:

A dedicated blockchain bridge allows core operations such as user registration, fund issuance, vendor registration/approval, and spending to be connected to a smart contract when blockchain integration is enabled.

Core Workflow

Fund Allocation

1. Government selects a beneficiary.
2. Government selects a fund category.
3. Government enters the amount.
4. Backend validates the beneficiary and amount.
5. A category-specific wallet is created or updated.
6. The allocation is recorded and audited.
7. The blockchain bridge can mirror the operation when enabled.

Fund Utilization

8. Beneficiary selects a fund category.
9. Beneficiary selects an approved vendor.
10. System verifies vendor approval.
11. System compares vendor category with fund category.
12. System checks available balance.
13. Valid transactions are approved.
14. Wallet balance is reduced.
15. Transaction and audit records are created.

Key Features

16. Purpose-Restricted Funds

Each fund is associated with a specific category, providing software-level enforcement of the intended purpose of government assistance.

17. Vendor Approval

Vendors have an approval status such as pending, approved, or rejected. Only approved vendors can receive transactions.

18. Automated Transaction Validation

The backend validates fund type, vendor category, vendor status, beneficiary role, transaction amount, and available balance before processing a transaction.

19. Audit Logging

Important operations such as registration, fund issuance, vendor actions, and spending generate audit records containing the user, action, target, details, and timestamp.

20. Public Transparency

The system provides aggregated statistics including total issued funds, total spending, transaction counts, active vendors, and category-wise utilization without returning personal information.

21. Dashboard Statistics

The backend calculates system-wide statistics including beneficiaries, registered/approved/pending vendors, spending, transactions, and category-level fund utilization.

22. Password Security

Passwords are not stored directly. The backend uses bcrypt-based password hashing before storing credentials.

23. Blockchain-Ready Design

The system contains a dedicated blockchain bridge so that core operations can interact with a smart contract rather than being limited to the local database.

Technology Stack

Frontend: Next.js, React, TypeScript

Backend: Python, FastAPI

API Validation: Pydantic

ORM: SQLAlchemy

Database: SQLite

Authentication: Passlib + Bcrypt

Server: Uvicorn

Blockchain Integration: Blockchain bridge / smart-contract interface

Development: Git & GitHub

Security and Accountability

TraceFund incorporates:

- Password hashing using bcrypt.
- Input validation through Pydantic models.
- Role-specific workflows.
- Vendor approval before transactions.
- Category-based transaction restrictions.
- Insufficient-balance protection.
- Audit logging.
- Public statistics without personal information.
- Blockchain integration capability.

These mechanisms work together to make unauthorized fund utilization more difficult and provide a traceable record of important operations.

Innovation

The main innovation of TraceFund is not simply digitizing government payments. It is embedding spending restrictions into the financial mechanism itself.

Instead of:

Government → Unrestricted Money → Beneficiary

TraceFund proposes:

Government → Purpose-Restricted Digital Fund → Approved Vendor → Verified Transaction → Audit Trail

This changes the system from a primarily trust-based model to a rule-enforced model. For example, if ₹5,000 is issued as a Medicine Fund, the system does not merely tell the beneficiary that it should be used for medicine. The transaction logic prevents the fund from being spent at an incompatible vendor.

Expected Benefits

- Greater accountability in welfare fund utilization.
- Reduced possibility of category-level misuse.
- Faster transaction validation.
- Easier auditing.
- Better visibility into government expenditure.
- Improved public transparency.
- Reduced dependence on manual verification.
- A foundation for future blockchain-based government funding systems.

Current Implementation Status

The repository contains a functioning prototype with implemented:

- User registration and login.
- Role management.
- Government fund allocation.
- Category-specific wallets.
- Vendor registration.
- Vendor approval/rejection.
- Restricted spending.

- Transaction records.
- Audit logs.
- Dashboard statistics.
- Public aggregate statistics.
- Blockchain bridge/API integration points.

The project should be presented as a prototype rather than a production government financial system. Remaining work includes stronger production authentication, additional blockchain deployment work, and completion of some dashboard/data components that currently use demonstration data.

Future Scope

24. Full deployment of smart contracts on a production blockchain.
25. JWT/session-based authentication and stronger authorization.
26. AI/ML-based fraud and anomaly detection.
27. Automated alerts for suspicious transactions.
28. Integration with government identity systems.
29. Multi-level government approval workflows.
30. Mobile application support.
31. Large-scale cloud deployment.
32. Advanced analytics and reporting.
33. Integration with real government payment and welfare databases.

Conclusion

TraceFund presents a practical approach to improving accountability in government fund distribution. By combining purpose-restricted digital funds, approved vendors, automated transaction validation, audit logging, transparency dashboards, and blockchain-ready infrastructure, the system demonstrates how technology can reduce opportunities for misuse while improving traceability.

The current implementation establishes the core transaction and accountability workflow as a working prototype, while its modular architecture provides a foundation for future blockchain, fraud-detection, authentication, and large-scale deployment enhancements.

TraceFund's central principle is simple: government assistance should not only be distributed — its intended use should be technically enforceable and independently traceable.